

A Pro-Active Defense Framework for IoT Systems

Elisa Bertino
Purdue University
bertino@purdue.edu

Hyunwoo Lee
KENTECH
hwlee@kentech.ac.kr

Mengdie Huang
Xidian University, Purdue University
huan1932@purdue.edu

Charalampos Katsis
Purdue University
ckatsis@purdue.edu

Zilin Shen
Purdue University
shen624@purdue.edu

Bruno Ribeiro
Purdue University
ribeirob@purdue.edu

Daniel de Mello
Purdue University
ddemello@purdue.edu

Ashish Kundu
Cisco Research
ashkundu@cisco.com

Abstract—As the number of interconnected IoT devices continues to skyrocket, so does the attack surface for malicious actors. IoT devices are often resource-constrained and deployed in diverse and challenging environments (e.g., critical infrastructures), making them susceptible to various vulnerabilities and attacks. Additionally, the attacks are often multi-step, which complicates the detection and mitigation efforts. Without proactive measures, these vulnerabilities can be exploited to devastating effects. In this paper, we propose a proactive defense framework for IoT systems security. We provide a high-level overview of the various components, encompassing training, detection, and attack management. Our approach begins with the design and implementation of *IoTEDef*, an innovative network intrusion detection system that employs per-step detectors based on the cyber kill chain and self-evolves to enhance the accuracy of the early step detectors. We present experimental results demonstrating significant enhancements in detector adversarial robustness following multiple rounds of evolution. We then briefly describe *IoTPredictor*, which utilizes real-time alerts generated by *IoTEDef* to predict potential attack next-steps, and *IoTAtM*, a system that leverages these predictions to take optimal actions for containing attacks using reinforcement learning. We also present *IoTDSCreator*, a tool designed to generate realistic datasets of multi-step attacks, serving as a valuable training data source for those components. We conclude by outlining future research.

Index Terms—Intrusion Detection, Attention Mechanisms, Reinforcement Learning, Adversarial Attacks, Edge Systems, IoT.

I. INTRODUCTION

Internet of Things (IoT) systems are the network of physical objects (or “things”), embedding electronics, software, and network connectivity. Such capabilities enable those objects to collect and exchange data [1]. IoT systems allow objects to be sensed and controlled remotely across existing network infrastructure, creating opportunities for more direct integration between the physical world and computer-based systems. IoT systems move computation and data storage closer to the sources of data and to end-user applications, especially when supported by edge computing capabilities that provide servers, local cloud, e.g. edge cloud, and various services. Such a decentralized architecture is expected to enhance data collection, fusion, and analysis capabilities, support the deployment of intelligent agents and applications in local contexts, and

improve response times, also because of progress in wireless communications (such as cellular network systems).

However, IoT systems face higher security risks than conventional computer systems [2]. IoT devices often have access to attackers’ targets such as sensitive data, cyber-physical systems, and user credentials [3]. Designing security techniques for IoT systems is challenging because IoT devices are often resource-constrained and thus hardly able to defend themselves. Often, they are not appropriately hardened, nor regularly patched, and not even managed according to security best practices, making them easy targets for attackers.

To provide a comprehensive approach to security, it is critical that the IoT systems in all their components be continuously monitored to detect anomalies or activities - indications of potential attacks, and if a potential attack is detected, actions should be executed to contain/stop the attack.

The goal of this project is to design and implement an AI-powered framework to address such a need. The envisioned framework consists of four components (see Figure 1): (a) *IoTDSCreator* - a tool to generate labeled datasets for multi-step attacks; (b) *IoTEDEF* - a self-evolving multi-agent anomaly detection system; (c) *IoTPredictor* - a tool that based on the results of the multi-agent anomaly detection and an attack knowledge base predicts possible next steps of the attack; and (d) *IoTAtM* - an attack management agent. The framework is complemented by an attack knowledge base consisting of attack graphs representative of important categories of attacks (e.g., ransomware, data breaches, sabotage, etc.) and graphs obtained by the labeled datasets generated by component (a). The knowledge base can be further enhanced by using widely used datasets containing information on different categories of attack vectors, attack steps, and CVEs.

The multi-agent anomaly detection system – the core of our framework is based on a kill chain consisting of the following steps: Reconnaissance, Infection, and Action. The detection system includes agents, refereed to as per-step detectors, specialized for detecting anomalies or activities related to attack vectors for each specific kill-chain step and a log system. Whenever an anomaly/attack-related activity is detected by one or more per-step detectors, the log is analyzed backward to determine events in the log that may represent infection points

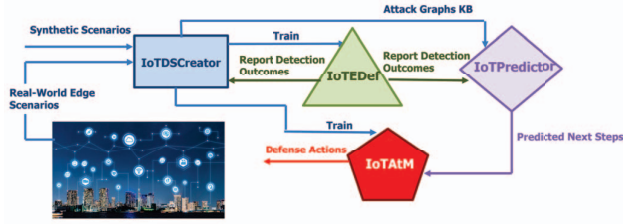


Fig. 1. High-level Architecture of the Pro-active Defense Framework and other attack activities. Since the time interval between those events can be long, the accuracy of such an analysis is critical. To address such an issue, we use the deep learning attention mechanisms designed for language translation to capture long-term dependency between the events designed for correlating events in IoT systems. Once a sequence of attack-related events is fully or partially reconstructed, the sequence is matched against the knowledge base of attack graphs to detect the ones with which the sequence has a partial match. It is important to notice that since we are dealing with ongoing attacks, the event log will only contain a partial view of the attack, and thus, the matches with the attack graphs are partials. Based on these matches, possible next steps of the attack can be predicted, and actions for containment undertaken. In addition, events in such a sequence can be used as samples to update the per-step detectors. This approach is particularly relevant to enhance the accuracy of the detection of early attack steps as it is challenging to detect early-phase attacks with both high precision and high recall. The reason is that to acquire a foothold in a target system, an attacker typically attempts to evade the detection systems by performing stealthy attacks (e.g., stealthy, distributed SSH brute-forcing) or exploiting completely unknown device/server vulnerabilities (e.g., zero-day attacks). To detect such attacks, the per-step detectors may classify all the suspicious or unknown patterns as anomalies, but it results in a high number of false positives. Therefore, continuous training of these per-step detectors is critical.

In this paper, we discuss the components of our framework. We start with IoTEDef, which has been implemented and evaluated. We then discuss the other components, which are still in the early stages of design. Throughout the discussion, we highlight research directions. Finally, we outline additional research directions.

II. SELF-EVOLVING DETECTION SYSTEM

In this section, we first describe the design principles and main architectural elements of the IoT Early Defense (IoT-EDef), as shown in Figure 2. We then discuss its robustness and report related experimental results. We refer the reader to [4] for more details and additional results.

A. Key Principles and Architecture of IoTEDef

IoTEDef is a network-based and self-evolving intrusion detection system that leverages the cyber kill chain to identify infection patterns and update the detector with the identified patterns to prevent the network from being subsequently

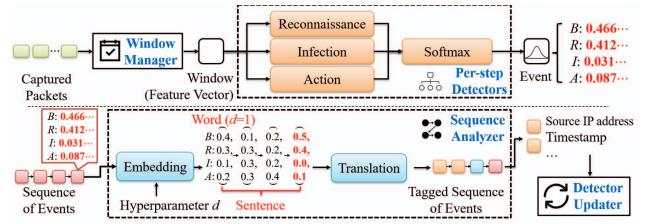


Fig. 2. Architecture of IoTEDef

infected again. The cyber kill chain is a framework that breaks down complex cyber attacks into mutually non-exclusive stages [5]. It shows the steps an attacker should complete one by one to achieve its malicious objectives. The subsequent steps could be executed once the previous steps are fulfilled.

IoTEDef models the attacks, based on the concept of the cyber kill chain, with the following three stages all of which are related to network communications: early-stage **Reconnaissance**, early-stage **Infection**, and later-stage **Action**. This attack model provides the precedence relationship between steps; thus, it is helpful to identify the patterns that belong to the previous step based on the observations of the patterns that belong to the later step. For instance if we detect the event of the action pattern, it means that there must be events in past that lead to the action pattern and those past infection events could be identified with the help of the action pattern.

Based on the above concept, IoTEDef consists of three per-step detectors, the sequence analyzer, and the detector updater. The modules work based on windows, each of which consists of flow features (e.g. the mean of the inter-arrival time between packets) extracted packets within a given period (e.g., 1 second).

Per-step detectors: The main role of the per-step detectors is to detect malicious patterns of the specific step from a given window. As IoTEDef is based on three attack steps, it consists of three per-detectors, called the reconnaissance detector, the infection detector, and the action detector. When a window is given, the detectors get the window as input separately and outputs if the window contains a reconnaissance pattern, an infection pattern, or an action pattern respectively. This maps a given window into the window that belongs to the specific step(s). The window can be labeled with multiple steps at this point. Therefore, instead of labeling the window with one specific step, we labels the window with the probabilities returned by the per-step detectors as their confidence. We call a window encoded with the probabilities an event.

Sequence analyzer: The main objective of the sequence analyzer is to identify the infection pattern from the observation of the action pattern. IoTEDef triggers this module to analyze the sequence of the events when it detects the action pattern with high probability. The sequence analyzer backtracks the sequence of the past events and finds the infection events that might lead to the action events. As this process relies on the relationship between different steps, we leverage the *attention mechanism* in deep learning that explicitly quantifies the level of the relationship between instances through the attention

weights. With the attention mechanism, the sequence analyzer returns a list of infection events related to the action event, which triggers the sequence analysis.

Detector updater: The detector updater aims to update the original infection detector with the identified infection events. In detail, this module retrains the infection detector based on the original training set and the identified infection events returned by the sequence analyzer.

B. Robustness

A critical issue when using machine learning (ML)-based detectors is represented by the robustness of the ML models used by these detectors. Robustness refers the ability of ML models to withstand evasion attacks. In what follows, we first introduce the attack models and the attacks considered in our evaluation, and then report selected initial results.

1) *Attack Models:* Evasion attacks vary depends on the assumptions about knowledge that the attacker has about the models to be bypassed. Such assumptions are referred to “attack models”. We focus on two attack models:

- **Black-box non-adversarial attack model:** We assume that attacks are launched from the Internet via remote network access. In other words, IoT devices are not compromised when initially deployed in the network, and the attacker lacks physical access to the IoT devices or direct access to the wireless network to which these devices are connected. We refer to attack samples created under this attack model as non-adversarial threat model evasion samples or **clean evasion samples**.
- **White-box adversarial attack model:** We assume that the attacker has extensive knowledge of the target model’s architecture, algorithms, and parameters. The attackers exploit vulnerabilities in DNN-based classifiers to create adversarial samples from clean ones. We focus on white-box adversarial attacks for two reasons. First, a model’s ability to withstand white-box attacks offers a thorough assessment of its overall robustness, indicating potential resilience against various attack types. Second, our preliminary experiments show that pre-evolved detectors are robust against black-box adversarial attacks like Boundary [6] and HopSkipJump [7].

2) *Adversarial Evasion Attack:* Unlike conventional evasion attacks based on packet segmentation, encryption, and obfuscation, adversarial evasion attackers craft the **adversarial evasion sample** x^* by adding a slight **adversarial perturbation** δ to the original evasion sample (aka clean evasion sample) [8], [9]. Techniques for generating adversarial evasion samples fall into two categories based on the attacker’s goal:

- **Untargeted Attack:** The attacker aims to find a δ maximizes the prediction error L of model f_θ without focusing on a particular target class, as shown in Equation (1).
- **Targeted Attack:** The attacker aims to find a δ that causes the model to misclassify the sample as the desired target class y_{target} , as shown in Equation (2).

$$\max_{x^*} L(f_\theta(x^*), y_{true}) = \max_{\|\delta\|_p \leq \epsilon} L(f_\theta(x + \delta), y_{true}) \quad (1)$$

$$\min_{x^*} L(f_\theta(x^*), y_{target}) = \min_{\|\delta\|_p \leq \epsilon} L(f_\theta(x + \delta), y_{target}) \quad (2)$$

For the binary classification, adversarial evasion samples (adversarial samples, for short) are usually only constructed from malicious samples. Thus, adversarial sample x^* can be generated according to Equation (1) with y_{true} set to malicious, or be constructed according to Equation (2) with y_{target} set to benign. We refer to the adversarial samples constructed from clean samples originally belonging to the infection category as **adversarial infection samples**.

3) *Adversarial Evasion Samples Generation:* To evaluate the robustness of IoTEDef against white-box adversarial attacks, we implement two well-known adversarial sample generation algorithms:

- **FGSM (Fast Gradient Sign Method):** FGSM [8] is a one-step adversarial attack that perturbs the input by adding a perturbation based on the sign of the gradients of the loss function w.r.t the input. However, FGSM may produce less effective perturbations in complex DNNs.
- **PGD (Projected Gradient Descent):** PGD [9] is an iterative version of FGSM. It applies FGSM multiple times with small per-step budget sizes and then projects the perturbations back to a predefined ϵ ball to ensure that the perturbations remain within a reasonable range. PGD is effective against a broader range of models.

When generating adversarial samples, we need to set the following budget parameters:

- **Perturbation Budget:** ϵ determines the maximum allowable perturbation on the clean sample x .
- **Step Size:** ϵ_{ite} determines the maximum allowable perturbation on the clean samples at each iteration.
- **Number of Iterations:** n_{ite} determines the maximum number of iterations a clean sample is allowed to be perturbed when making an adversarial sample.

We refer to the original infection test set as the **clean infection test set**, and the set of adversarial evasion samples generated from the infection samples in the clean infection test set as the **adversarial infection test set**. Note that the clean infection test set contains clean benign samples and clean infection samples, while the adversarial infection test set only contains adversarial infection samples.

4) *Experimental Results and Analysis:* We evaluate the robustness of the evolved infection detector in two adversarial attack and defense scenario settings. In these scenarios, the attacker conducts N rounds of adversarial evasion attacks. The clean infection test set is denoted as X , and the pre-evolved infection detector is denoted as M_0 .

- **Adversarial Setting I:** The attacker creates adversarial infection samples using the latest evolved target infection detector and the same clean infection test set in each attack round. Notably, since the adversarial samples used in each attack round are crafted based on different target models, they have distinct adversarial data distributions.
- **Adversarial Setting II:** In each new attack round, the adversary crafts adversarial samples using the latest evolved

detector and a clean infection test set different from the previous round. To facilitate the experiments, we divide the clean infection test set X into N sub-datasets $X_1, X_2, \dots, X_N, X_{N+1}$. For N rounds of evolution, we split the original clean test set into $N + 1$ sub-test sets. The number of benign and malicious samples in each sub-test set remains the same.

We answered the following research questions in adversarial setting I and II based on experimental data.

- ① Does the False Negatives (FN) of the infection detector on white-box adversarial samples decrease after multiple rounds of evolution?
- ② How many rounds of evolution can the detector be robust to white-box adversarial attacks?
- ③ Does the False Positives (FP) of the infection detector on black-box non-adversarial samples (clean data) increase after multiple rounds of evolution?

In adversarial setting I, we have three main findings.

① From Figure 3 we observe that the pre-evolved infection detector (M0) is highly susceptible to PGD attacks, as all adversarial infection samples are mistakenly classified as benign. However, after self-evolution, the updated infection detector's recall significantly improves, demonstrating that the self-evolution strategy effectively enhances its ability to identify deceptive malicious samples.

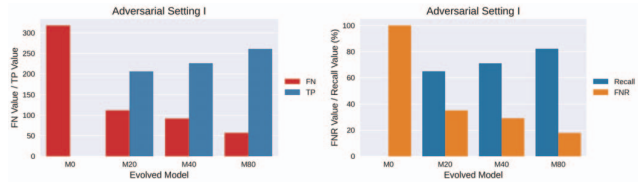


Fig. 3. Average Performance of The Evolved Infection Detector on Adversarial Samples (AdvSetting I).

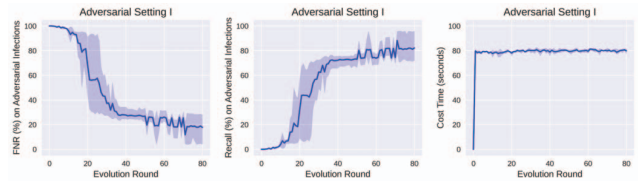


Fig. 4. Impact of the Number of Evolution Rounds on Robustness against White-box Attack (AdvSetting I).

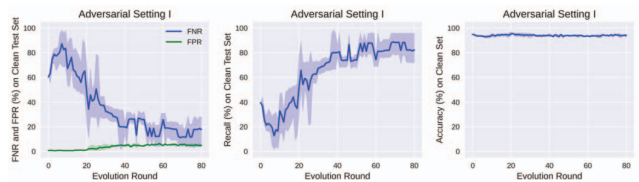


Fig. 5. Average Performance of The Evolved Infection Detector on the Clean Test Set (AdvSetting I).

② From Figure 4 we can see that the detector's robustness consistently improves as the number of evolution rounds

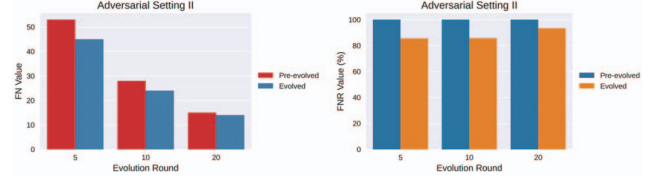


Fig. 6. Average Performance of the Evolved Infection Detector on Adversarial Samples (AdvSetting II).

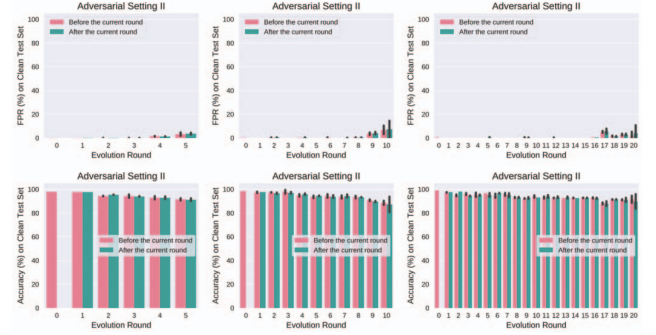


Fig. 7. Average Performance of The Evolved Infection Detector on the Clean Test Set (AdvSetting II).

increases. Generally, within 30 rounds, the accuracy of the detector on white-box adversarial samples can exceed 50%, and the FNR starts to stabilize at 20% around the 40th round.

③ From Figure 5 we observe that the increase in the False Positive Rate (FPR) of the evolved infection detector on the clean test set is minimal compared to the sharp reduction in the False Negative Rate (FNR). This suggests that the evolved infection detector has a stronger ability to identify truly malicious samples, including those that are non-adversarial.

In adversarial setting II, we draw two main conclusions.

① From Figure 6 we observe that the pre-evolved detector has difficulty in identifying adversarial infection samples designed to masquerade as benign. But after 5, 10, and 20 rounds of evolution, the average recall of the infection detector significantly improves, indicating that evolution helps reduce FN and increase True Positives (TP).

② From Figure 7 we see that the self-evolution strategy leads to only a minor rise in FPR across various retraining rounds. This, along with the detector's accuracy on the clean test set, shows that the evolved detector not only becomes more robust against white-box adversarial attacks but its performance on black-box non-adversarial samples is not greatly affected.

C. Research Directions

1) *Defense against Real-time and Domain-constrained Adversarial Attacks:* Adversarial evasion attacks against network intrusion detection systems (NIDSs), such as IoTEDef, are updated rapidly. In the future, our research in this area will focus on two key aspects.

First, we aim to improve the robustness of the NIDS against real-time adversarial attacks. Most adversarial attacks create adversarial samples one by one from clean ones. Efficiently

generating these adversarial samples relies heavily on the algorithm used. We have evaluated IoTEDEF's robustness against FGSM and PGD attacks. However, PGD, while powerful, involves complex iterative processes requiring significant computational resources. This can introduce network packet delays, negatively impacting the original samples' goal. To address real-time adversarial threats, we will develop more efficient algorithms to create adversarial perturbations for live network traffic and explore robust detection methods.

Second, we plan to enhance the robustness of IoTEDEF against domain-constrained adversarial attacks. Adversarial samples were originally proposed for images but have recently been applied in various domains, including text, audio, malware, and network traffic. Unlike the isotropic nature of image feature vectors, the feature vectors of network traffic are anisotropic, and the numerical types and value ranges between feature dimensions are different. Consequently, when adapting adversarial sample generation algorithms designed for the image domain to the network traffic domain, we need to make adjustments to align with these unique attribute constraints. We will apply tailored perturbations to different traffic feature dimensions, ensuring that the modified adversarial traffic samples maintain their functionality while minimizing their perturbations. Then we will design an extended version of IoTEDEF for these practical adversarial traffic scenarios.

2) *Additional Adversarial Attack Approaches*: The generation of adversarial attacks is fundamental to the progress of adversarial defense robustness research. Efficient and effective adversarial attacks are crucial for reliable evaluation of defenses [10]. In the case of NIDSs, generating adversarial attacks typically involves modifying malicious packets to bypass detection. However, the generation of adversarial attack samples for NIDSs must satisfy two requirements [11]: the manipulated packets should still cause attacks, while at the same time the manipulated packets should not result in network disconnection. Therefore, the manipulation of malicious packets to generate adversarial attack samples needs careful adjustment. Given such requirements, the attacker is constrained to modify specific features of the traffic, such as the timestamp [12] and packet length. To deal with this situation, some approaches [13], [14] leverage the Particle Swarm Optimization (PSO) algorithm to modify the traffic packets and evade the detection of NIDSs. Subsequent to generating the adversarial attack samples, the re-training can significantly enhance the robustness of the NIDS. An interesting research direction is to determine which packet features (or combinations of features) can be modified and which ones cannot, and then compare the effectiveness of PSO-based generation of malicious samples with other approaches, such as PGD [9].

III. TRAINING DATA GENERATION TOOL

Advancing ML-powered security solutions for IoT requires high-quality datasets to train robust detection mechanisms. Yet, the current dataset landscape has several limitations. First, many datasets are outdated or fail to well reflect the intricacies

of multi-step attacks within IoT ecosystems. Second, they have a limited perspective on attacks; for example, some datasets exclusively offer network-focused insights, like pcap files, while neglecting information about the events occurring at the end systems, such as IoT devices or host machines. This partial view restricts the holistic detection capabilities, particularly at end-systems. Last, the context in which a dataset is generated holds paramount importance. For instance, a dataset may only include data on compromised IoT devices transmitting malicious packets but not malicious packets targeting the device themselves.

Therefore, as part of our framework, we design *IoTDSCreator*, a tool to create labeled datasets of multi-step attacks for IoT ecosystems. Given a scenario description, IoTDSCreator automatically builds an emulation-based network and executes a multi-step attack. It then collects data from both the network (e.g., switches) and end systems (e.g., IoT devices and hosts) and generates a labeled dataset where the labels are the cyber kill chain attack steps.

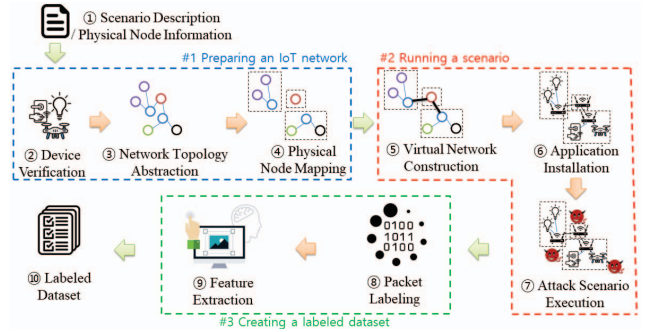


Fig. 8. The IoTDSCreator execution steps.

A. IoTDSCreator Execution Steps

IoTDSCreator is organized according to multiple steps (see Figure 8):

(1) **Scenario description**: The scenario is a Python script that uses the API provided by the IoTDSCreator to orchestrate the network environment and perform the attacks. IoTDSCreator allows users to model a large variety of IoT device attributes, encompassing both software stacks and hardware specifications, including the kernel, file system, and executed applications.

(2) **Device verification**: This step verifies the scenario description to make sure that the modelled device characteristics are consistent with the scenario described in Step (1). For instance, if ARM TrustZone technology is specified, the device configuration must be in accordance with an ARM processor. Effectively, this step operates as a scenario “compiler”, pinpointing erroneous configurations.

(3) **Network topology abstraction**: In this step, the topology described in the scenario is converted to an abstracted virtual network represented as a graph. The nodes represent interconnected devices such as switches, routers, and IoT devices, while the edges represent the links between them.

(4) Physical node mapping: To ensure scalability, IoTDSCreator facilitates the extension of the virtualized network across multiple physical machines. In this distributed setting, each machine executes an *Agent* that awaits the directives from IoTDSCreator regarding the virtualized network to be established. Agents also transmit additional information, such as machine status and execution progress.

(5) Virtual network construction: Once the agents complete the subnetwork deployments on their corresponding machines, IoTDSCreator connects them seamlessly to the network, and the packets are transmitted through network tunnels.

(6) Application installation: Upon completing the network configuration, devices proceed to install the required applications as delineated in the scenario. These applications can encompass tasks such as monitoring temperature or humidity within a physical space.

(7) Scenario execution: The scenario of the multi-step attack is executed. At the same time, IoTDSCreator captures the generated network packets from several network points, such as the IoT gateway or from the attacker's network interfaces.

(8) Packet labeling: The attack packets are mapped to the corresponding attack step as per the kill chain, which includes Reconnaissance, Initial Access, Infection, and Action.

(9) Feature extraction: The captured data undergoes feature extraction, with appropriate encoding techniques applied based on semantics and value range, such as one-hot encoding.

(10) Labeled dataset generation: The IoTDSCreator tool undertakes data refinement tasks like eliminating duplicate packet entries and generates a dataset of the executed attack scenario.

B. Tool Enhancements

The development of the scenario description (step 1) requires a deep understanding of the network environment as well as familiarity with the IoTDSCreator APIs. The former requires expertise in network configuration, including tasks such as link and network interface setup. The latter involves translating the intended configuration into a carefully crafted scenario code using the IoTDSCreator API. Consequently, we aim to explore abstractions that simplify the user's role to only specify the attack steps and their connections.

One such abstraction that well represents the steps of an attack is the concept of *attack graph* [15]. An attack graph consists of nodes and directed edges connecting them. In essence, each node represents a state within the attack, and the edges represent the conditions facilitating the attacker's transition from one state to another. In various works [16]–[18], a state corresponds to a reported CVE from credible sources like national vulnerability databases. The edges represent the sequence of exploitation steps, whether within the same device or across interconnected devices, all aimed at achieving the attacker's objectives.

Therefore an interesting research direction is to design techniques to automatically generate the scenario code for IoTDSCreator, given an attack graph. To this end, we outline two initial approaches.

Approach 1: This approach involves exploring graph-to-text natural language techniques [19]–[21]. These methods typically leverage sequence-to-sequence (seq2seq) text generation [22] using machine learning models such as Long Short-Term Memory networks (LSTM), Convolutional Neural Networks (CNN), or Generative Pre-trained Models (GPT).

This approach requires careful consideration of three key factors. First, training these machine learning models may demand a substantial volume of data, depending on the chosen model. Notably, GPT-based models present an attractive option due to their extensive pre-training on vast datasets, potentially requiring fewer data for fine-tuning. Second, there is the matter of generating the training data itself. Given that IoTDSCreator is a novel framework with its own API, the selected model will need to be trained on numerous graph-to-code pairs to grasp the nuances of generating IoTDSCreator-specific code from attack graphs. Finally, there is the matter of code correctness. It is thus critical to develop techniques that guarantee that the generated code is free of errors and its ability to execute the scenario in IoTDSCreator effectively. An interesting idea is to incorporate code correctness checks within the model verification and testing procedures. Consequently, the training phase evolves into an iterative process, where the model (re-)trains until specific performance criteria (e.g., code correctness) are achieved on both seen and unseen data.

Approach 2: In this approach, we refrain from utilizing ML for direct code generation. Instead, we employ ML to extract essential information from attack graphs, followed by a deterministic code generation process. The concept revolves around traversing the graph from source to sink, with a focus on extracting context from each node and edge. For instance, we aim to infer the type of vulnerability, whether it pertains to software, hardware, or the operating system. In addition, we extract pertinent details such as version information for software, hardware components, or the operating system (OS). Furthermore, we categorize the actions associated with the edges, such as command execution, session establishment, or network packet generation. Achieving this information extraction relies on the capabilities of language models, including Named Entity Recognition and GPT. The extracted data can then be encoded into a structured format, such as JSON, before being fed into a code generator. The code generator effectively maintains a mapping of all feasible extracted information to specific sequences of function calls within the IoTDSCreator tool. For example, if the vulnerability description (i.e., graph node) mentions a particular IoT application, the translator generates the appropriate code (using the map) to install and execute the application on an IoT device. This approach is expected to reduce the reliance on ML, particularly because a model like GPT may already prove adequate for the required information extraction. However, it requires a greater level of engineering. A thorough study of the vulnerability listings is critical to accurately determine how much and what information we can extract from a given node or edge through ML. Furthermore, it is crucial to ensure that the mapping maintained by the code generator is comprehensive and up-to-

date, encompassing all potential scenarios. Developing such a comprehensive mapping may require significant time and effort.

IV. ATTACK STEP PREDICTION TOOL

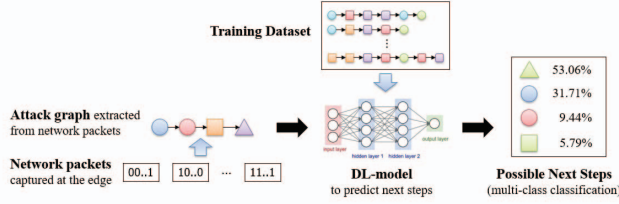


Fig. 9. Architecture of IoTPredictor

IoTPredictor (see Figure 9) is a deep learning-based tool that, based on the anomalies detected by IoTEDef per-step detectors, predicts the possible next steps in a multi-step attack. Precise prediction is challenging as it requires a prediction model to be trained with various scenarios. We address such a challenge with IoTDSCreator that can generate many kill chain based attack datasets, from which attack graphs can be generated to obtain an attack graph knowledge base.

It is, however, important to notice that the per-step detectors are not perfect and thus they may misclassify network packets, events, activities and may also be bypassed by the attacker. Therefore based on the single-step detectors, the defense system can, in many cases, only build a partial graph and then match this partial graph against the set of graphs in the knowledge base, using also similarity techniques. Similarity techniques are useful to determine whether similar attacks have been seen before, or the attack is completely new. Of course the defender partial graph may match more than one graph in the knowledge base and thus the attacker can obtain from the match different next steps that the attacker could do. Therefore a relevant research issue is to design approaches for associating probabilities to such possible next attack steps, risks and so forth. Based on these matches the defender can then decide how to react to the attack (using the IoTAtM agent in our defense framework) and also understand what the goal of the attack could be. An interesting research is to use some specialized LLM to predict next steps.

V. ATTACK MANAGEMENT AGENT

The attack management agent, IoTAtM, is based on reinforcement learning (RL) techniques, as these techniques are particularly suitable for optimal decision making.

A Reinforcement learning (RL) framework [23] (see Figure 10) is a probabilistic state transition environment where state transitions are caused by actions executed by an agent and every state-action pair (s, a) is assigned a reward value r given by a reward function $R(s, a)$. A RL agent traverses the environment according to a policy π that selects an action to execute in a given state for policy parameters θ and receives the rewards accrued over all the state transitions that have happened according to θ . In a Q learning framework, the goal

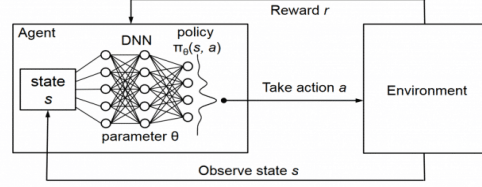


Fig. 10. Deep Q Learning Environment

is to find the optimal policy which maximizes the cumulative reward through a function $Q(s, a)$ that estimates the expected cumulative reward that will get at the end of an episode if the current state is s , the agent executes a and follows the learnt policy π_θ . In a deep Q RL system, a deep neural network, referred to as DQN, is used to determine the optimal Q function using the following temporal difference equation:

$$Q_t(s, a) = Q_{t-1}(s, a) + \alpha[R(s, a) + \gamma \max_{a'} \{Q_t(s', a')\} - Q_{t-1}(s, a)]$$

where Q_t is the current estimation of the Q function, Q_{t-1} is the previous estimation of Q after $t - 1$ steps of training. The estimated next state and action are denoted by s' and a' , respectively. The learning rate (α) determines to what extent newly acquired information overrides old information. The discount factor (γ) determines the importance of future rewards.

The application of an RL framework to attack management entails addressing several challenges. The first is modeling the reward function R as it may require estimating the costs of different actions to be executed in response to attacks and risk parameters, that have to be combined in some mathematical expression (see [24] for an example). An alternative solution is the use of a ML-based predictor, which would need to be trained in advance. The second challenge is the scalability of the RL framework due to possibly very large numbers of states, which may lead to the state explosion problem. A research direction is related to techniques for splitting up large state spaces into smaller ones through state-space decomposition. The third challenge is the training of RL agents. There are three main approaches to training: (i) on-policy, by which the agent interacts with the environment using the latest learned policy, and then uses that experience to improve the policy; (ii) off-policy, by which the agent interacts with the environment to collect experience samples but also re-uses samples from older policies, thus enhancing efficiency; and (iii) offline, by which the agent is trained on a static dataset of fixed interactions and thus does not have to interact with the environment. A research direction is related to customize and combine these approaches, especially the first two, among the agents according to their roles and the environments in which they are deployed. For example, even with (the simplest) approach (iii), the trained agents can adapt by using for example smaller numbers of explorations or transfer learning [25].

VI. CONCLUSIONS

Developing machine learning models targeting secure IoT systems presents multiple challenges, some of which are addressed by our framework. Large AI models are difficult to deploy in the resource-constrained settings typical of IoT devices due to their significant power and data requirements. We believe the solution is not falling back to simple models, rather, we must leverage the capabilities of large resource-hungry models in a way amenable to IoT deployment.

A future research direction considers generalizing IoTEDef's approach to the challenges of ever-changing data distributions (distribution shifts). IoTEDef can be categorized as a knowledge distillation procedure for semi-supervised learning: A large-scale model (sequence analyzer) is trained using a limited set of labeled data. This large model subsequently guides the training of smaller models (per-step detectors) using large unsupervised datasets. Currently, this knowledge distillation process (and also our adversarial defense training) is initiated by system administrators. That is, system administrators decide when the detector updater should run. This model update is necessary since machine learning models fail under distribution shifts, when the data observed in test is significantly different from the one used in training. Ideally, however, this decision must be done at the IoT device, automatically. This requires the development of a distribution shift detector that determines the optimal timing and which data should be given to IoTEDef's detector updater for retraining the per-step detectors by the sequence analyzer. We also note that this framework can be applied to detecting whether the IoTEDef's edge model needs to be adversarial-hardened by the detector updater through our adversarial methods when the adversary starts an evasion attack (incurring a distribution shift).

ACKNOWLEDGMENT

The work reported in this paper has been supported by Cisco Research, NSF under grant 2112471, and Xidian University.

REFERENCES

- [1] H. Lee, A. Mudgerikar, A. Kundu, N. Li, and E. Bertino, "An infection-identifying and self-evolving system for iot early defense from multi-step attacks," in *ESORICS 2022 - 27th European Symposium on Research in Computer Security, Copenhagen, Denmark, September 26-30, 2022, Proceedings, Part II*.
- [2] E. Bertino and N. Islam, "Botnets and internet of things security," *IEEE Computer*, vol. 50, no. 2, pp. 76–79, 2017.
- [3] D. Dingee, "Iot, not people, now the weakest link in security," <https://devops.com/iot-not-people-now-the-weakest-link-in-security/>, January 2019, (Accessed on 05/13/2021).
- [4] M. Huang, H. Lee, A. Kundu, X. Chen, M. A., N. Li, and E. Bertino, "Ariotedef: Adversarially robust iot early defense system based on self-evolution against multi-step attacks," in *Under submission*. ACM, 2023.
- [5] Gartner, "Addressing the cyber kill chain: Full gartner research report and lookingglass perspectives," 2016, (Accessed on 06/03/2021).
- [6] W. Brendel, J. Rauber, and M. Bethge, "Decision-based adversarial attacks: Reliable attacks against black-box machine learning models," in *Proceedings of International Conference on Learning Representations (ICLR)*, 2018.
- [7] J. Chen, M. I. Jordan, and M. J. Wainwright, "Hopskipjumpattack: A query-efficient decision-based attack," in *Proceedings of IEEE Symposium on Security and Privacy (S&P)*. IEEE, 2020.
- [8] I. J. Goodfellow, J. Shlens, and C. Szegedy, "Explaining and harnessing adversarial examples," in *Proceedings of International Conference on Learning Representations (ICLR)*, 2015.
- [9] A. Madry, A. Makelov, L. Schmidt, D. Tsipras, and A. Vladu, "Towards deep learning models resistant to adversarial attacks," in *Proceedings of the International Conference on Learning Representations (ICLR)*, 2018.
- [10] G. Sriramanan, S. Addepalli, A. Baburaj *et al.*, "Guided adversarial attack for evaluating and enhancing adversarial defenses," *Advances in Neural Information Processing Systems*, vol. 33, pp. 20 297–20 308, 2020.
- [11] F. Pierazzi, F. Pendlebury, J. Cortellazzi, and L. Cavallaro, "Intriguing properties of adversarial ml attacks in the problem space," in *2020 IEEE symposium on security and privacy (SP)*. IEEE, 2020, pp. 1332–1349.
- [12] Y. Sharon, D. Berend, Y. Liu, A. Shabtai, and Y. Elovici, "Tantra: Timing-based adversarial network traffic reshaping attack," *IEEE Transactions on Information Forensics and Security*, vol. 17, pp. 3225–3237, 2022.
- [13] E. Alhajjar, P. Maxwell, and N. Bastian, "Adversarial machine learning in network intrusion detection systems," *Expert Systems with Applications*, vol. 186, p. 115782, 2021.
- [14] D. Han, Z. Wang, Y. Zhong, W. Chen, J. Yang, S. Lu, X. Shi, and X. Yin, "Evaluating and improving adversarial robustness of machine learning-based network intrusion detectors," *IEEE Journal on Selected Areas in Communications*, vol. 39, no. 8, pp. 2632–2647, 2021.
- [15] O. Sheyner, J. Haines, S. Jha, R. Lippmann, and J. M. Wing, "Automated generation and analysis of attack graphs," in *Proceedings of the 2002 IEEE Symposium on Security and Privacy*. IEEE, 2002, pp. 273–284.
- [16] M. U. Aksu, K. Bicakci, M. H. Dilek, A. M. Ozbayoglu, and E. i. Tatli, "Automated generation of attack graphs using nvd," in *Proceedings of the Eighth ACM Conference on Data and Application Security and Privacy*, 2018, pp. 135–142.
- [17] Z. Fang, H. Fu, T. Gu, P. Hu, J. Song, T. Jaeger, and P. Mohapatra, "Iota: A framework for analyzing system-level security of iots," in *2022 IEEE/ACM Seventh International Conference on Internet-of-Things Design and Implementation (IoTDI)*. IEEE, 2022, pp. 143–155.
- [18] J. Payne, K. Budhraj, and A. Kundu, "How secure is your iot network?" in *2019 IEEE International Congress on Internet of Things (ICIoT)*. IEEE, 2019, pp. 181–188.
- [19] L. F. Ribeiro, M. Schmitt, H. Schütze, and I. Gurevych, "Investigating pretrained language models for graph-to-text generation," *arXiv preprint arXiv:2007.08426*, 2020.
- [20] L. Song, Y. Zhang, Z. Wang, and D. Gildea, "A graph-to-sequence model for amr-to-text generation," *arXiv preprint arXiv:1805.02473*, 2018.
- [21] R. Koncel-Kedziorski, D. Bekal, Y. Luan, M. Lapata, and H. Hajishirzi, "Text generation from knowledge graphs with graph transformers," *arXiv preprint arXiv:1904.02342*, 2019.
- [22] D. Bahdanau, K. Cho, and Y. Bengio, "Neural machine translation by jointly learning to align and translate," *arXiv preprint arXiv:1409.0473*, 2014.
- [23] R. S. Sutton and A. G. Barto, *Reinforcement learning: An introduction*. MIT press, 2018.
- [24] A. Mudgerikar, E. Bertino, J. Lobo, and D. Verma, "A security-constrained reinforcement learning framework for software defined networks," in *ICC 2021 - IEEE International Conference on Communications, Montreal, QC, Canada, June 14-23, 2021*.
- [25] Z. Zhang, A. Mudgerikar, A. Singla, K. Leung, E. B. D. Verma, K. Chan, J. Melrose, and J. Tucker, "Reinforcement and transfer learning for distributed analytics in fragmented software defined coalitions," in *SPIE Conference on Artificial Intelligence and Machine Learning for Multi-Domain Operations Applications, April 2021*.